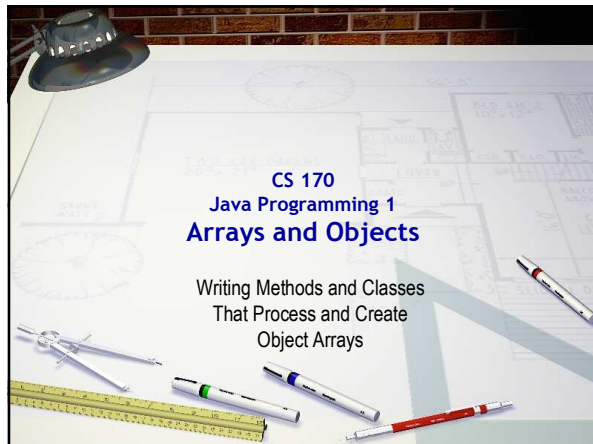
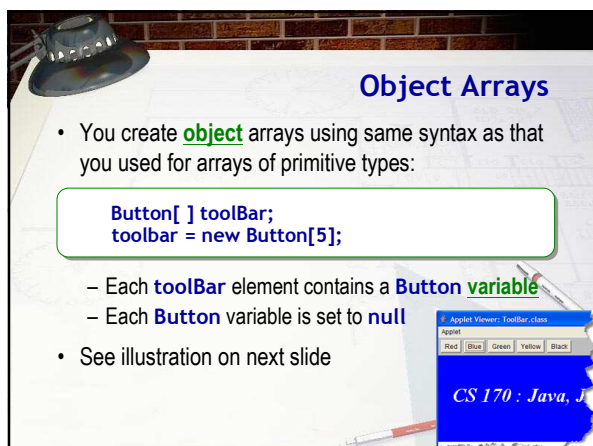
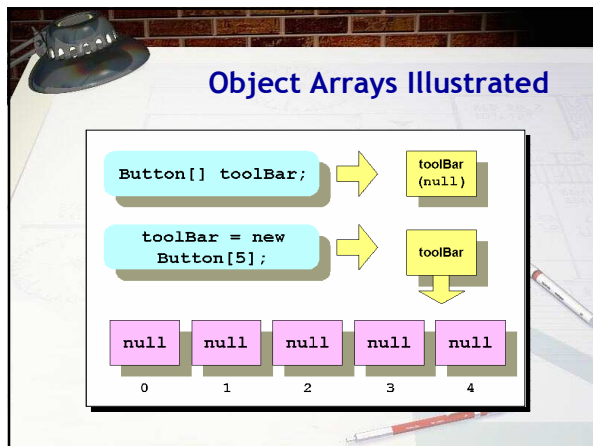








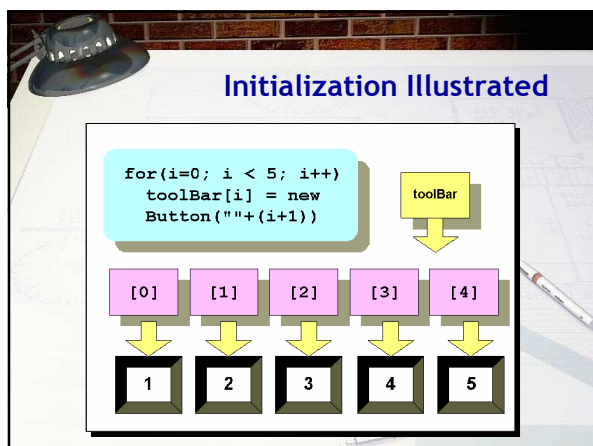


Initializing Object Elements

- Before you can make use of the **toolBar** array, you must create the actual object **elements** separately

```
int len = toolBar.length;  
for (int i = 0; i < len; i++)  
{  
    toolBar[ i ] = new Button(" " + (i + 1));  
}
```

- See illustration on the next slide



Express Yourself

- Open **CardDeckArray** in the ic18 project
- **Exercise 2:** Create an array variable named **deck** that can refer to an array of **Card** objects
- **Exercise 3:** Create a method named **newDeck()** that returns an array of **Card** objects
 - Create a temporary **Card** array
 - Use nested loops: **Suit.values()** and **Rank.values()**
 - Construct a **Card** object inside the inner loop
 - Add it to the next element in the array
 - Return the temporary array

Express Yourself

- **Exercise 4:** Locate **actionPerformed()** and the code that adds a new deck to the table. Add code to:
 - Call the **newDeck()** method and assign the returned value to the **deck** variable from Exercise 2
 - Set the **cardsRemaining** to the **length** of the deck
- **Exercise 5:** Locate the **mouseClicked()** method and the block that deals a card from the deck pile
 - Flip the current card in the deck face up
 - Move the current card from the deck to the card pile
 - Run and make sure it works. Shoot me a screenshot.

Moving Elements

- Array elements also often need to be “rearranged”
 - **Sorting** puts elements in order (ascending, descending)
 - **Shuffling** places elements in random order
- Example: Pick three “lotto” balls
 - Numbers must be randomly selected between 1 and 50
 - No number can appear twice



```
Random rand = new Random();  
int n1 = rand.nextInt(50) + 1;  
int n2 = rand.nextInt(50) + 1;  
int n3 = rand.nextInt(50) + 1;
```

Shuffling

- Best way is to put each "ball" in an array like this:

```
05: {  
06:     Random rand = new Random();  
07:  
08:     // Initialize the elements  
09:     int lotteryBalls[] = new int[50];  
10:     for (int i = 0; i < 50; i++)  
11:         lotteryBalls[i] = i + 1;  
12: }
```

- Then, **shuffle** the array and pick the first three:
 - 1. Loop through the entire array (like first pattern)
 - 2. Generate random index value each time
 - 3. Swap value at current index with value at random
- See code example on next slide

The Swap Pattern

- The shuffle algorithm in code

```
12: // Shuffle them  
13: for (int i = 0; i < 50; i++)  
14: {  
15:     int j = rand.nextInt(50);  
16:     int temp = lotteryBalls[i];  
17:     lotteryBalls[i] = lotteryBalls[j];  
18:     lotteryBalls[j] = temp;  
19: }  
20:  
21:
```

- Note use of the **swap** or **exchange** pattern
 - Create a temporary variable and save the first value
 - Copy the second value into the first variable
 - Copy the saved value into the second variable

A Subtle Problem

- Former sort looks good, but isn't truly random
 - Problem is that permutation is not really random
- Better algorithm: Fisher-Yates (or Knuth) shuffle
 - Take last card, exchange with any other card
 - Take next-to-last exchange with any other card, **except** for the very last card
 - Continue on until the first card has been swapped.
- Exercise 6:** Add the **shuffle()** method to your code
 - Call the method from **actionPerformed()**
 - Run and make sure it works. Shoot a screenshot.

Reversing Elements

- **Exercise 7:** open **ArrayPracticeExercises** and locate the **main()** method. Given the following declarations

```
Scanner scan = new Scanner(System.in);
String inStr = scan.nextLine();
char[] phrase = new char[inStr.length()];
```

- Fill the array **phrase** with the characters from **inStr**
- Reverse the characters in **phrase** by swapping the first element with the last, the second element with the next to last, and so on until the array is reversed.
- Print the array. Run and shoot a screen-shot.

Shifting Elements

- Shifting means moving each element left or right
 - "Rotate" wraps first or last item around to other end
 - Use nested loops to shift multiple times

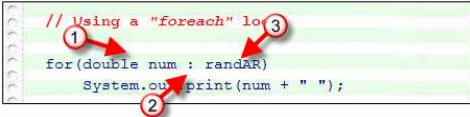
```
int first = a[a.length - 1]; // save last element
for (int i = a.length - 1; i > 0; i--)
{
    a[i] = a[i-1]; // shift each element right
}
a[0] = first; // wrap around the last element
```

Express Yourself

- **Exercise 8:** The previous slide showed you how to rotate an array right by one element. Open the file **ArrayPracticeExercises** and complete the method **rotateLeft()**. This method modifies the array by left-shifting each by 1 (instead of right-shifting), so that **{6, 2, 5, 3}** becomes **{2, 5, 3, 6}**. When you've completed the method, run the JUnit **TestRotateLeft** and make sure that all the tests are green.
- Complete and test **rotateLeftN()** by calling the **rotateLeft()** method **n** times. Run the JUnit tests.

Non-modifying Algorithms

- Java 5 introduced a simplified version of the **for** loop
 - Known informally as the **foreach** loop
 - Visit every element and process each value
 - Can't modify elements or use the loop counter
 - Think of it as a **read-only** version of the **for** loop
- The syntax elements



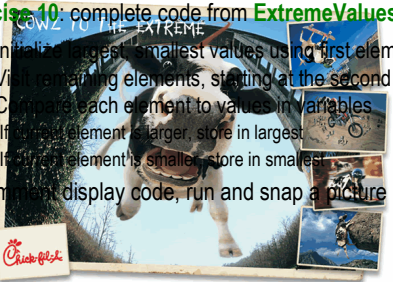
```
// Using a "foreach" loop
for(double num : randAR)
    System.out.print(num + " ");
```

Sum, Average, Count

- Exercise 9: SumAvgCount** example applet
 - 1. Store the length of the array (**len**)
 - 2. Initialize elements using **Math.random() * length**
 - 3. Use new-style loop to calculate **sum** and **avg**
 - Create and initialize **sum** before the loop
 - Add each element to **sum** in loop body
 - Divide by **len** and assign to **avg** after loop
 - 4. Count numbers larger or smaller than average
 - Store in variables **larger** and **smaller**
- Run and snap a picture

Extreme Values

- Find largest and/or smallest of a sequence of values
- Exercise 10: complete code from ExtremeValues**
 - 1. Initialize largest/smallest values using first element
 - 2. Visit remaining elements, starting at the second
 - 3. Compare each element to values in variables
 - If the element is larger, store in largest
 - If the element is smaller, store in smallest
- Uncomment display code, run and snap a picture



Partially-Filled Arrays

- Often you'll only want to process a **portion** of an array
 - Can't use the "natural" loop bounds
 - Can't use the foreach version of the for loop
- Most common reason? Only some of the data is valid
 - Known as a "**partially-filled**" array
 - Since array size can't be changed you must **plan for the worst case** and make array large enough to hold maximum desired number of data elements
 - Keep track of number of elements actually entered

The InputArray Program

- A program to enter a variable number of input values
 - Keep track of capacity and size separately



- Exercise 11:** Open **InputArray** program.
 - Create an array to hold 100 **double** values.
 - Also create **int size** and **capacity** variables
 - Save and shoot a pic of code (errors now gone)

Partially-Filled Array Input

- Program uses a sentinel loop to get user's input
 - Prompts for input (note use of **capacity**)
 - Keeps going while **size < capacity**
 - Read a number into a local variable
 - Check if the number is our exit (sentinel) value

```

26 // Get the input from the user
27 System.out.println("Enter up to " + capacity +
28                     " positive real numbers.");
29
30 System.out.println("Enter 0 or a negative number to quit.");
31
32 while (size < capacity)
33 {
34     // Read a number
35     double num = scan.nextDouble();
36
37     // If the user wants to exit, then leave
38     if (num <= 0)
39         break;
40

```

Partially-Filled Array Output

- Use traditional **for** loop: process **only** valid elements
 - Use the **size** variable to control the loop

```

46:
47: // 8. Display results
48: System.out.print("ar = (");
49: if (size == 0)
50: {
51:     System.out.println(")");
52: }
53: else
54: {
55:     for (int i = 0; i < size; i++)
56:         System.out.print(" " + ar[i]);
57:     System.out.println(")");
58: }
59:
60: Enter up to 100 positive real numbers.
61: Enter 0 or a negative number to quit.
62: 25.2 3.5 99 101 25.353 75
63: 19 25 -2
64:
65: ar = (25.2, 3.5, 99.0, 101.0, 25.353, 75.0, 19.0, 25.0)

```

Inserting or Removing

- Suppose we want to "insert" items in correct order
- Assume 4th item entered is 3.25, like this:



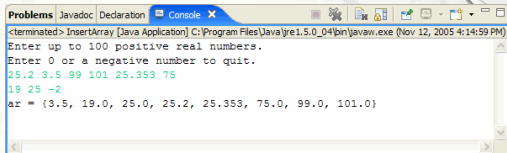
- 1. Locate first number larger than 3.25 (7.1)
- 2. Move remaining elements one to the right
 - Must start at the last element or we'll overwrite data
- 3. Copy element into its final resting place
- Only makes sense in context of partially-filled arrays

Insertion: Finding Position

- Open the **InsertArray** example program
 - Similar to **InputArray**, but we'll keep numbers in order
- Step 1:** Find location where new element should go
 - Declare position variable (**pos**) outside of loop
 - In loop, compare each element to input number
 - Exit the loop when a larger element is found
 - Use the **break** statement
 - Will point to **size** if no elements larger
- Exercise 12:** add code for finding position
 - Shoot a screen-shot of your code

Insertion: Move then Add

- To “open up a slot” move elements to the right
 - Shift each element from pos to end right one
 - Note you need to start at last valid element
 - Continue while index is > than insertion point
- **Exercise 13:** add loop to open up space



```
Problems | Javadoc | Declaration | Console
<terminated> InsertArray [Java Application] C:\Program Files\Java\jre1.5.0_04\bin\javaw.exe (Nov 12, 2005 4:14:59 PM)
Enter up to 100 positive real numbers.
Enter 0 or a negative number to quit.
25.2 3.5 99 101 25.353 75
19 25 -2
ar = {3.5, 19.0, 25.0, 25.2, 25.353, 75.0, 99.0, 101.0}
```

Removing Elements

- Works almost identical to inserting elements
 - Locate position of element to be removed
 - Copy items to right of element to “close up” the “gap”
- Must make sure that your loop goes left-to-right
 - Bounds must be < size-1 (see element copied from)

```
for (int i = delPos; i < size-1; i++)
    ar[i] = ar[i + 1];
```

Finish Up

- In-class exercise: submit **before** you leave today!
 - Drop into Q:\faculty5\sgilbert\cs170\submissions
- Finish reading Chapter 7, start Chapter 9
 - We'll also cover some GUI material from back in Chapters 4 and 5. Use the book for reference.